

Utilizing Weight Space Learning for Data-Free Model Editing

Anonymous Authors¹

Abstract

The growing availability of model checkpoints has enabled Weight Space Learning (WSL), a deep learning paradigm that uses model parameters as a data modality. While WSL has been used for learning functions of neural network weights, like accuracy prediction, its potential for direct model editing remains underexplored. One useful case of model editing is machine unlearning, where the reliance on original or target data to forget currently limits applicability in strict privacy and legacy systems. We illustrate the potential of utilizing WSL for data-free model editing, through examples of *zero-shot* unlearning and performance improvement. We train a metanetwork to capture the optimization landscape of the *Small CNN Model Zoo*, predicting class recall with high accuracy (mean $R^2 \approx 0.842$). By backpropagating a loss through the metanetwork with respect to its input weights, we obtain directions in weight space that selectively edit target-class performance. On held-out models, this achieves up to 52 percent recall drops on forget classes while preserving retain-class accuracy.

1. Introduction

In AI interpretability research, neural networks are usually studied through the lens of their input-activation-output behavior. Recent work highlights that insights into their mechanistic functionality can be gained by examining their parameters directly (Braun et al., 2025; Bushnaq et al., 2025). This perspective treats the weights of a trained model as a vector in a high dimensional space, the *weight space*. The core hypothesis is that information about what a neural network ‘knows’ or what it is capable of is encoded into its parameters, and that this information can be extracted. This is in line with the machine learning subfield called *weight*

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

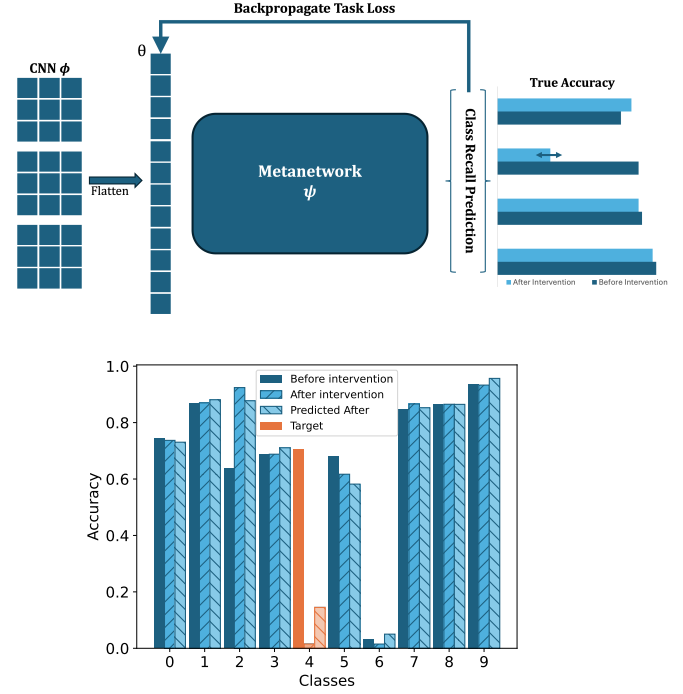


Figure 1. **Top:** A collection of CNN models are flattened and used to train a metanetwork to predict multivariate class recall. A task specific loss function w.r.t. the metanetwork **inputs**, i.e. the CNN parameters, can be used to edit performance of similar but before unseen CNN models. **Bottom:** an effective example result of our method. We unlearn class 4 in a Fashion-MNIST classification CNN: Small CNN Zoo model #86 which was not yet seen by the metanetwork.

space learning (WSL), which establishes neural network weights itself as a new data modality, driven by the vast and ever-growing number of publicly available neural network models. A collection of neural network weights used for weight space learning is called a *model zoo*. *Metanetworks* are trained on these model zoos to infer what a model has learned without requiring explicit access to the original training methods, data or model architecture. Using metanetworks to study large collections of models shows that weight configurations can reveal training data, tasks, hyper-parameters and performance of a model (Eilertsen et al., 2020; Unterthiner et al., 2021).

Simultaneously, the need for model editing is becoming more relevant. After (pre-)training, some functionality of

a model might need change. Improving performance on some downstream task, removing bias or specific training data. Partly due to legal requirements such as the *right to erasure* established in the EU’s General Data Protection and Regulation, or ethical demands such as model alignment, we might aim to remove the influence of specific data or tasks from a trained model. A straightforward option is to edit the dataset and retrain the model, but the scale of many current state-of-the-art models in use makes this impractical. This need has driven *machine unlearning*: selectively removing learned information without full retraining. However, most methods assume access to training or forget data impractical when such data is proprietary or restricted.

Weight interpolation shows that to some extent these edits can be made in the parameters of a model directly (Ainsworth et al., 2022). Ilharco et al. (2022b) show that a task or concept can be isolated by the direction in weight space a model moves when finetuning on that specific task. Thus, *task vectors* in weight space can be identified that point towards improvement on a specific task. These task vectors can also be used to perform task arithmetic in weight space, such as negation for unlearning, or recombination for transfer learning.

We introduce a weight space learning perspective on model editing. We propose to train a metanetwork that predicts performance on specific tasks based on the weights of a large set of models from the same model-family. A well-trained metanetwork implicitly encodes the optimization landscape; its input gradients can thus serve as task vectors, pointers in weight space for targeted editing.

In practice we propose to backpropagate the gradient of a task specific loss function through the regressor with respect to its inputs: the model weights. This allows for data-free gradient descent for iteratively editing model performance on any quantifiable and differentiable task. We demonstrate this through on two class-specific editing tasks: forgetting and learning. Our method is summarized in Figure 1. Specifically, we train a metanetwork to predict per-class accuracy of CNN models the Small CNN Zoo model collection (Unterthiner et al., 2021), a more fine-grained task than overall performance prediction. We show that the metanetwork predicts class recall with high accuracy ($R^2 \approx 0.84$), disentangling class-specific functionality. On held-out models never seen during training, the metanetwork provides actionable editing signals, through the backpropagation of a loss function w.r.t. the metanetwork’s inputs: the original CNN model weights.

As a generic approach that operates solely on weight vectors without task-specific design, we do not expect to outperform dedicated unlearning methods that leverage model inference or synthetic data generation. Rather, our aim is to demonstrate that a WSL metanetwork encodes sufficient

structure about the optimization landscape to provide an actionable editing signal, thus establishing a proof of concept for weight-space model editing.

Our contributions are as follows:

1. We show a metanetwork can be trained to predict the per-class recall with high performance.
2. We develop a **data-free** model editing algorithm that uses gradients backpropagated through this metanetwork to adjust the weights of **unseen** neural networks.
3. We demonstrate effectiveness on two editing tasks: targeted forgetting and performance improvement, showing selective modification of class-specific behavior.
4. We extend the Small CNN Model Zoo, by computing per-class accuracy (class recall) and make this dataset extension publicly available.

This work is, to the best of our knowledge, the first demonstration of adding a loss function to the output of a metanetwork and using input attribution to make changes to the weights of that input model instance, with the goal of editing functionality in a pretrained model.

2. Related Work

2.1. Weight Space Learning

Two pioneering works in weight space learning are by Unterthiner et al. (2021) and Eilertsen et al. (2020). Both successfully demonstrate using a model zoo to predict model properties with a meta-network using model weights as inputs. The works differ mainly in their prediction target: Unterthiner et al. (2021) predict model performance from weights, whereas Eilertsen et al. (2020) predict different training algorithm hyperparameters from weights. Using meta-classifiers, they demonstrate that even small subsets of weights encode rich information about datasets, optimizers, and activations. We extend the methodology of Unterthiner et al. (2021) by using a metanetwork to predict the per-class recall of the Small CNN Zoo models. To the best of our knowledge this is the first demonstration of such fine-grained performance prediction by a metanetwork.

In recent years, impressive steps have been made to address symmetries, scaling, and generalization to diverse feedforward architectures in WSL (Schürholt et al., 2021; 2022a;b; Navon et al., 2023; Zhou et al., 2023; Lim et al., 2024; Kalogeropoulos et al., 2024; Schürholt et al., 2024; Knyazev et al., 2023; Wang et al., 2025; Liang et al., 2025). A more detailed overview of the developments of the field is given in Appendix A.

Concurrent to work presented here, [Moulin et al. \(2025\)](#) have taken a similar approach of using a trained metanetwork to inform a loss function of which the gradient is backpropagated w.r.t. the input weights; albeit as an element within a larger training framework, and used **during** optimization—whereas we aim to directly edit pre-trained models. Specifically, they show that WSL can also be used to guide generalization in RL. The authors train a model zoo of RL policies on two toy problems and then train a metanetwork to predict the generalizability of these policies. This metanetwork is used to inform an additional loss component to the PPO algorithm ([Schulman et al., 2017](#)) during training of new policies. This leads to improved generalization in the policies trained with this metanetwork-guided loss function versus those trained without.

[Rangel et al. \(2024\)](#) presented a weight space learning perspective on machine unlearning. Self-supervised learning (SSL) metanetworks can create *hyper-representations* of model weights. Their method achieves unlearning by generating unlearned models, using diffusion transformer hypernetworks, which are trained on models *with* knowledge of the target class and models *without* knowledge of the target class. While impressive, this method relies on access to *target models* which serve as a ground truth for the desired performance: models trained from scratch on a dataset where the data to forget is excluded. Instead, we aim to present a method for editing pre-trained models where access to an example edited model is not available, thus having improved practical implications.

2.2. Model Editing & Machine Unlearning

A seminal work that introduced the concept of finding task vectors in the parameter space of a model and using those to edit a pre-trained model is [Ilharco et al. \(2022a\)](#). They show that a task or concept can be unlearned by negating the direction in weight space a model moves when finetuning on that specific task. They also show improvement on tasks and task transfer through other task-vector arithmetic. For forgetting image classification tasks via negation they report the accuracy of a pre-trained Vision Transformer to drop by 45.8 percentage points on average on target classes they wish to forget, with little loss on the control task. They apply two baselines; finetuning via gradient ascent, i.e. training in the *wrong* direction, and applying a random vector on a per-layer basis of the same magnitude as the task-vector.

Post-hoc model editing—and specifically unlearning—is increasingly important due to growing pre-training costs and regulatory emphasis on data privacy and ownership. Most unlearning methods assume access to original training data, or at minimum the forget set ([Nguyen et al., 2025](#)). [Chundawat et al. \(2023\)](#) argue that these assumptions are often

impractical¹ and propose *zero-shot* unlearning: “*requiring no training samples or information related to the training process*”. Some recent work has relaxed this definition to permit forget-set access ([Foster et al., 2024a](#)). However, our approach adheres to the strict original definition by [Chundawat et al. \(2023\)](#). As a solution they propose two methods: Error minimizing-maximizing noise and gated knowledge transfer (GKT). Both methods require full access to a working model, whereas our weight space learning based method only needs the weight vector. Furthermore, their best method requires training a generator from scratch per model; our method front-loads the computation into metanetwork training, then editing is just a few gradient steps for every model you wish to edit after pre-training.

Recently, [Mishra et al. \(2025\)](#) have proposed data-free unlearning for CLIP-style multimodal models ([Radford et al., 2021](#)). They exploit the joint representation space by computing text embeddings from class names via the text encoder, and orthogonalizing image features away from these embeddings. In contrast, our metanetwork-based approach is architecture-agnostic in principle: given a suitable metanetwork, the editing procedure itself is independent of the target model’s structure. As discussed in Section 2.1 and Appendix A, metanetworks have been successfully applied across diverse architectures, and recent developments such as graph metanetworks ([Lim et al., 2024](#)) can handle variable architectural inputs by design.

3. Preliminaries

Let $\mathcal{C}$ be a set of K classes. In the case of a model editing task, the class we aim to influence, the target class, is denoted as c . The set of remaining classes on which we wish to preserve performance are denoted $\mathcal{C}_r = \mathcal{C} \setminus c$, the *retain* or *control* set. Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ be a dataset consisting of N sample-label pairs, where each data sample $x_i \in \mathbb{R}^D$ is associated with a one-hot vector label $y_i \in \mathbb{R}^K$. $\mathcal{D}_t$ is defined as the set of data corresponding to target class label c , $\mathcal{D}_r$ is that to $\mathcal{C}_r$. Let $\phi_\theta(x) : \mathbb{R}^D \rightarrow \mathbb{R}^K$ be a neural network parameterized by $\theta \in \mathbb{R}^W$ trained to classify samples by mapping inputs to class probability distributions. $\mathbb{R}^W$ is called the weight space. We denote a model zoo $\Phi = \{\phi_{\theta_j}(x)\}_{j=1}^M$ as a collection of M models sharing similar model architecture but with varying parameters θ_j .

4. Method

In this section we introduce the model-editing algorithm, the metanetworks, and the model zoo used in our experiments. The core of our method is shown in Algorithm 1. Several implementations through a set of loss functions and stopping criteria are discussed.

¹see section 8 for further discussion

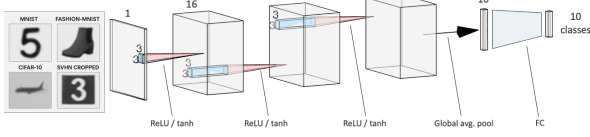


Figure 2. The architecture of the models in the Small CNN Zoo. It consists of three convolution layers with a 3x3 kernel and 16 output channels, followed by a global average pooling flattening operation and a fully connected layer to 10 output logits. The input images have one channel. In total this results in 4970 trainable parameters

4.1. A Metanetwork for Data-free Editing

We train a Multi-Layer Perceptron (MLP) metanetwork $\psi(\theta; \omega) : \mathbb{R}^W \rightarrow \mathbb{R}^K$, parameterized by ω , to perform multivariate regression on the per-class recall of the models in Φ . To demonstrate the usefulness of the representation ψ has learned of the task performance of a model zoo Φ , we attempt to edit held-out models from Φ based on information guided by ψ . Notably, we perform gradient descent on the metanetwork’s *input*, rather than its own parameters, i.e. the weights θ of the original model ϕ . The trained metanetwork ψ serves as a differentiable proxy for the performance of ϕ , enabling data-free model editing. The proposed algorithm is described in Algorithm 1.

Algorithm 1 Weight Space Model Editing

Require:

Loss function $f(\psi(\theta; \omega)) : \mathbb{R}^W \rightarrow \mathbb{R}$ (differentiable),
 input model $\phi(x; \theta)$ with initial parameters $\theta_0 \in \mathbb{R}^W$,
 trained metanetwork $\psi(\theta; \omega)$,
 step size $\eta > 0$,
 binary stopping criterion $s(\cdot)$,
 tolerance $\varepsilon > 0$

```

1:  $\theta \leftarrow \theta_0$ 
2: while not  $s(\theta, \varepsilon)$  do
3:    $\mathbf{g} \leftarrow \nabla_{\theta} f(\psi(\theta; \omega))$   $\triangleright$  Compute gradient w.r.t. input
4:    $\theta \leftarrow \theta - \eta \mathbf{g}$   $\triangleright$  Gradient step
5: end while
    
```

4.2. Loss function

For unlearning, we want to move ϕ_{θ} in the direction of an expected decrease of performance on $\mathcal{D}_t$ by $\psi(\theta)$, and to maintain performance on $\mathcal{D}_r$. The loss function used for this task is labeled **Boost Loss**:

$$f = \psi_c(\theta) - \beta \sum_{i \in C_r} \psi_i(\theta). \quad (1)$$

For the performance improvement experiment, we want to increase the performance on $\mathcal{D}_t$, but aiming to decrease

the performance on $\mathcal{D}_r$ is not considered a realistic use case, so merely flipping the sign on the Boost Loss seems nonsensical for this experiment. Rather, in our loss function labeled **Simple Loss** we only focus on target class c :

$$f = \psi_c(\theta). \quad (2)$$

4.3. Stopping criterion

We cannot expect ψ to be faithful on the entire weight space. Preliminary experiments have shown that after a number of iterations ψ can leave its ‘trust region’: it can become worse at predicting the class recall of ϕ accurately. Thus we cannot rely on convergence of the loss as a stopping criterion for Algorithm 1. Since the proposed intervention is data-free there is no direct evaluation method for determining if and when the intervention is sufficiently successful. We thus propose ψ to serve as a proxy for determining performance of the edited model, as a stopping criterion for determining when to halt Algorithm 1.

Absolute Accuracy Prediction When the predicted accuracy on the target class drops below or rises above a set threshold (depending on the task) the algorithm is halted.

Relative Accuracy Prediction Because the initial accuracy differs between specific instances of ϕ , we may want to look at the predicted **relative** difference. For example, we can halt the algorithm once the metanetwork predicts a 50% accuracy drop over the original accuracy.

5. Experimental setup

To showcase model editing, we perform two specific experiments on previously unseen CNNs; **Unlearning**: decreasing performance of a pre-trained ϕ on $\mathcal{D}_t$, and **uplearning**: increasing the performance of a pre-trained ϕ on $\mathcal{D}_t$, both while aiming to preserve performance on the remaining data $\mathcal{D}_r$.

5.1. Small CNN Zoo

For our experiments we use the Small CNN Zoo as Φ (Unterthiner et al., 2021). This dataset contains training checkpoints for 120,000 small underparameterized CNN image classifiers, each trained on one of four datasets: MNIST (Lecun et al., 1998), Fashion-MNIST (F-MNIST) (Xiao et al., 2017), CIFAR-10 (Krizhevsky et al., 2009), or SVHN (Netzer et al., 2011). Training was done with a wide sweep of hyperparameters to achieve a zoo of models with a wide range of varying performance, resulting in 4 CNN collections each containing checkpoints of approximately 30,000 CNNs over 86 epochs. Figure 2 shows a visualization of the architecture of the CNNs. We partition Φ into disjoint

sets Φ_{train} , Φ_{val} , Φ_{test} containing 15,000, 7,500, and 7,500 models respectively.

We extend the Small CNN Zoo by reconstructing and re-evaluating the networks for class recall, or per-class accuracy. The per-class accuracy for all 120,000 models is reported for each epoch stage in the canonical split (early, middle, final) and made available at [redacted for anonymity].

For the unlearning experiment we filter out extremely poor performing or incomplete models from the evaluation set. All models that have 0% accuracy in any class or less than 10% accuracy in the target class are excluded from the evaluation set.

5.2. Metanetwork details

ψ takes the flattened CNN weight vector θ as input. The network consists of 5 hidden layers with 256 ReLU-activated units each and a dropout rate of 0.03. The final layer is a linear transformation followed by a sigmoid activation to produce multi-output regression values. These hyperparameters were chosen to maximize R^2 on a validation set and were not found to be too sensitive to tuning. We include CNN checkpoints from early, middle, and final training stages. This exposes ψ to a broader range of weight configurations, which we find marginally improves prediction faithfulness when weights are modified during editing.

5.3. Score functions

Like Ilharco et al. (2022a) we report accuracy on a target ($\mathcal{D}_t$) and the average accuracy on control dataset ($\mathcal{D}_r$). In addition, we measure the success of the interventions by 2 different scores: **Target difference** - Direct change in class accuracy of class c . **Max difference** - Change of the class c relative to the maximum change of the classes in $\mathcal{C}_r$. This is a strict metric: if at least one of the untargeted classes has had a change larger than that of the target class, the max difference score will be negative.

5.4. Baselines

We compare against three baselines that test whether the metanetwork provides a meaningful editing signal: a random edit of the same magnitude as our edits, and for the unlearning experiment a naive finetuning solution and an ideal baseline using finetuning on the retain set $\mathcal{D}_r$.

Random vector Like Ilharco et al. (2022a) we evaluate against a baseline of per-layer random vector edits. We compute the per layer magnitude of our proposed edit $\tau^{(L)} = \|\theta^{(L)} - \theta_0^{(L)}\|$, then draw a random vector $v^{(L)} \sim \mathcal{N}(0, I)$ from a standard Gaussian distribution and scale it to $\tau^{(L)}$, giving $\tau_{\text{random}}^{(L)} = \frac{v^{(L)}}{\|v^{(L)}\|} \tau^{(L)}$, a random edit

per layer. These are concatenated and added to θ_0 yielding a weight-vector that can be used in the same way as θ .

Gradient ascent As in Golatkar et al. (2020); Tarun et al. (2023); Ilharco et al. (2022a) we create a baseline by finetuning exclusively on the target task, but negating the loss function. Effectively unlearning via gradient ascent.

Retain-set finetuning For unlearning, a standard approximate unlearning baseline is finetuning on the retain set $\mathcal{D}_r$ (all data except the forget class) for a few epochs (Golatkar et al., 2020; Foster et al., 2024b). Following Foster et al. (2024b), we use 5 epochs with the original training hyperparameters.

5.5. Experiments

We trained 10 separate metanetworks per to predict per-class recall per all four Φ collections. We report the average coefficient of determination R^2 in addition to MSE and MAE for this multivariate regression task, as in Unterthiner et al. (2021). One of the 10 metanetworks is selected at random, and used to edit models of Φ_{test} , for every class between 0-9 as c . For all experiments a step size η of 0.1 is chosen.

Runs that do not terminate after 2000 steps, as well as runs that do not move the model more than 1 arb. unit Euclidean distance in weight space are excluded from the reported results. These post-hoc filters sort out models where the method was clearly ineffective, not in a positive or negative sense.

Unlearning For unlearning we aim to reduce accuracy on a target class whilst retaining accuracy on the control classes. As a loss function we use the Boost Loss function (Equation 1) with $\beta = 0.1$ and the *relative accuracy prediction* stopping criterion at with a threshold at 60% accuracy reduction. Motivation for this is explained in the Section 6.

Uplearning For uplearning we aim to improve accuracy on the target class whilst retaining accuracy on the control classes. We use the Simple Loss function (Equation 2) and the absolute accuracy prediction stopping criterion at 90%. This threshold is set high to maximize uplearning efforts for demonstration.

6. Results

We find consistent high performance for predicting per class recall of Φ_{test} . These results are reported in Table 1. The resulting metanetworks function highly consistent, an error margin of ± 1 standard deviation over the 10 trials for the R^2 is reported.

dataset	train		validation		R^2
	mse	mae	mse	mae	
MNIST	0.012	0.050	0.014	0.056	0.922 ± 0.001
F-MNIST	0.013	0.060	0.016	0.069	0.893 ± 0.003
CIFAR-10	0.012	0.062	0.015	0.076	0.696 ± 0.012
SVHN	0.009	0.047	0.012	0.055	0.855 ± 0.025

Table 1. Average of 10 metanetworks performance for predicting class recall on validation sets per image-dataset. Uncertainty reported for R^2 .

An unbiased sample of resulting predictions on individual models, and the effect of Algorithm 1 for unlearning, is reported in Appendix D.

6.1. Unlearning

Figure 3 shows results of an exploratory experiment for tuning relative stopping threshold. Unlearning is attempted for 1494 MNIST validation set models on target class 4, varying relative stopping threshold. This is set out against a baseline of random vector edits of the same magnitude. We find that increasing stopping threshold increases the effect of our intervention on target class, but also deteriorates the control classes. To balance these effects we choose a relative stopping threshold of 60% for further experiments.

After evaluation, runs that did not terminate after 2000 steps and runs that did not move the model significantly in weight space, i.e. less than 1 a.u. Euclidean distance, are excluded from the results reported here.

Boxplots of the target difference and max difference scores per class are reported in Figure 4 and Figure 5 respectively. In almost all cases a median positive result of the intervention is achieved. This is evident by a net positive median target difference, green lines in the boxplots. The intervention is not always isolated. Figure 5 shows that on average

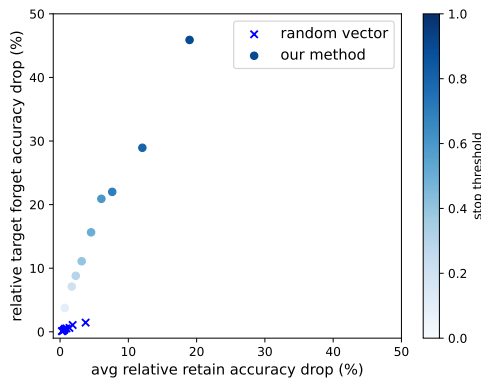


Figure 3. Forget class versus Retain classes median relative accuracy drop (%) for 1494 MNIST validation set models.

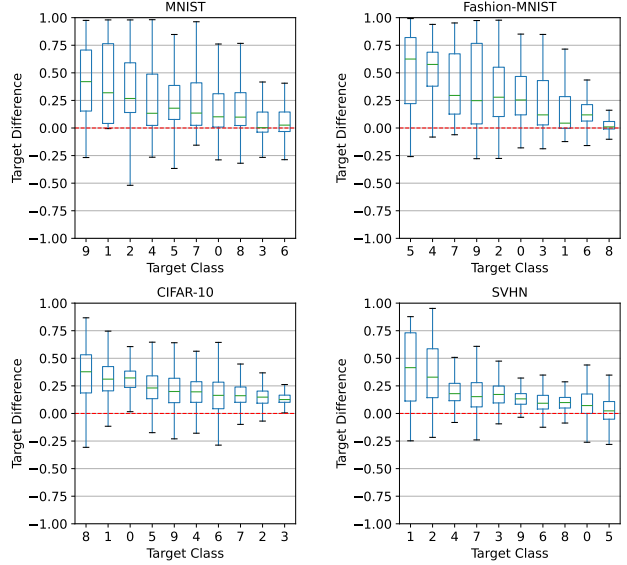


Figure 4. Boxplots of target difference by target class after intervention for MNIST, Fashion-MNIST, CIFAR-10, and SVHN on the validation set. Ordered by mean target difference; outliers omitted. Upwards of the red dotted line marks successful intervention.

the target class moves with at least one other class.

Table 2 reports median unlearning results aggregated over all 10 separate target classes. The gradient ascent baseline achieves the lowest target recall ($\mathcal{D}_t$) but catastrophically damages control group ($\mathcal{D}_r$) performance. The random vector edits show that random edits of the same magnitude as our proposed have a slight and uniform deteriorating effect over both target and control groups. Our method balances target group reduction with control group preservation.

6.2. Uploading

Table 3 reports median uploading results aggregated over all 10 separate classes per dataset. Again we find that the intervention on average is successful for all datasets, except for SVHN. On the SVHN models our method performs similar to a random vector edit. The relatively poor performance might be explained by an overly optimistic stopping criterion. Most CIFAR-10 and SVHN models in the Small CNN Zoo dataset have low overall performance, which might make 90% performance on a class unreachable.

7. Discussion

We find that CNNs can be edited by applying edits obtained via input attribution of a metanetwork. Our experiments highlight the potential of editing models by applying task-vectors obtained via backpropagation of an arbitrary loss function with respect to the input of a metanetwork.

Figure 4 and 5 show a large spread in the effectiveness of our methods so far. We find that our method has a varying

Table 2. Unlearning baseline comparison across datasets. Median accuracy over all target classes after intervention. Target ($\mathcal{D}_t$) recall should decrease; Control ($\mathcal{D}_r$) recall should remain high. Number of algorithm evaluations: MNIST: 27021, F-MNIST: 32086, CIFAR-10: 26952, SVHN: 13838.

Method	MNIST		F-MNIST		CIFAR-10		SVHN	
	$\mathcal{D}_t \downarrow$	$\mathcal{D}_r \uparrow$	$\mathcal{D}_t \downarrow$	$\mathcal{D}_r \uparrow$	$\mathcal{D}_t \downarrow$	$\mathcal{D}_r \uparrow$	$\mathcal{D}_t \downarrow$	$\mathcal{D}_r \uparrow$
Before	0.87	0.89	0.78	0.75	0.42	0.38	0.48	0.43
Retain finetune	0.01	0.89	0.00	0.77	0.00	0.40	0.00	0.45
Gradient ascent	0.00	0.13	0.00	0.15	0.00	0.12	0.00	0.11
Random vector	0.82	0.82	0.74	0.72	0.35	0.33	0.36	0.35
Our method	0.61	0.73	0.36	0.65	0.13	0.34	0.29	0.35

Table 3. Results for uplearning. Median accuracy over all target classes after intervention. Target ($\mathcal{D}_t$) recall should increase; Control ($\mathcal{D}_r$) recall should remain. Number of algorithm evaluations: MNIST: 2632, F-MNIST: 1239, CIFAR-10: 8841, SVHN: 2214.

Method	MNIST		F-MNIST		CIFAR-10		SVHN	
	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r \downarrow$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r \downarrow$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r \downarrow$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r \downarrow$
Before	0.69	0.72	0.70	0.74	0.36	0.34	0.39	40
Random vector	0.64	0.65	0.66	0.67	0.18	0.24	0.17	0.24
Our method	0.72	0.62	0.81	0.57	0.61	0.19	0.20	0.23

effect per model, and there is a bias between specific classes: some classes are effected better than others. In practice this means that there are many examples for which metanetwork-based model editing works very well (Figure 1 (bottom) shows example where the unlearning edit was exceptionally effective), but there are also a good number of models in Ψ_{test} on which the method does not work or even shows a negative effect. Furthermore, early experimentation showed that while not having a significant effect on the population-level results, the results of metanetwork-based model editing could differ largely based on specific ϕ - ψ combinations. As of yet it is unclear what causes the inter-model variance, and the inter-class bias. Further research is needed to interpret and explain the sources of this variance. It might partially be explained by the nature of the Small CNN Model Zoo. The models in this model zoo were trained with a wide array of varying hyperparameters, some well outside the range of common values for effective training. This was done with the explicit purpose of creating a very diverse population of CNN models. Although this is favorable for training robust metanetworks which have seen a wide range of different models (as discussed in Section 5.2), future work could look into creating more controlled populations of test models, for better interpretable experimentation. The CNNs are also well in the underparameterized regime with only 4970 trainable parameters. This could be a cause for the difficulty of isolating edits to a specific class as shown in Figure 5.

Crucially, both the unlearning procedure and the metanetwork training can be achieved without access to CNN input data. For the metanetwork training a dataset of trained models and their evaluation statistics is sufficient, and the

unlearning algorithm is solely based on a loss function on the metanetwork predictions. This means that metanetwork-based model editing can be a truly data-free methodology. This makes the approach applicable in realistic settings where original training data are unavailable, restricted, or legally protected.

8. Outlook

This work is mainly presented as a proof of concept, a non-exhaustive list of potential promising avenues for future work are discussed here.

Single model approach The same methodology may be used to edit single models locally. Metanetwork training could be done on a dataset based on noise perturbation of a model parameters. A metanetwork could then be used to edit that specific model using methodology described here.

More fine-grained concepts In this work, we used per class accuracy as the predictor variable of the metanetwork. This shows that metanetworks may be used to predict model properties or functionality that are not directly evident from simple inspection of the weights. This methodology could be applied in an even more fine-grained matter as long as some predictor value may be defined and evaluated, e.g. predicting confusion between classes or performance on a specific subset of the data. This could potentially allow for metanetworks to be used for de-biasing models.

Other model zoos Novel insights and dedicated efforts have driven the development of larger model zoos cover-

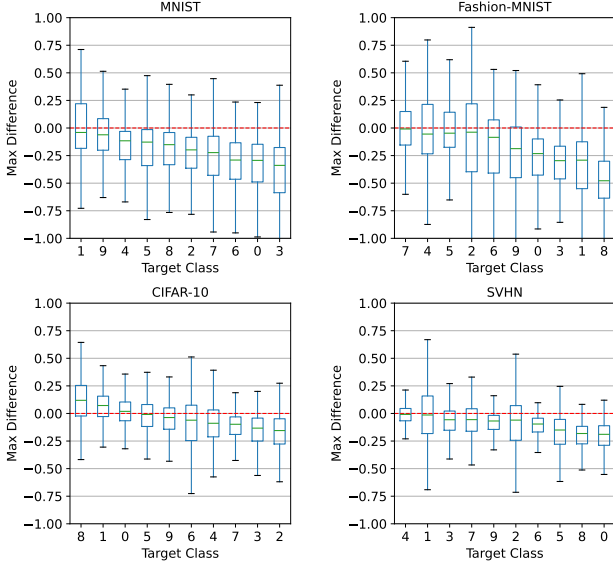


Figure 5. Boxplots of max difference between target class and all other classes after intervention for MNIST, Fashion-MNIST, CIFAR-10, and SVHN on the validation set. Ordered by mean target difference; outliers omitted. Upwards of the red dotted line marks successful intervention.

ing multiple architectures, which may be more suitable for robust weight space learning (Schürholt et al., 2022b) and therefore can aid in effective model editing by metanetworks. Future work should investigate how larger zoos and different input model architectures can improve performance, generalization, and stability in weight-space model editing.

Other metanetwork architectures In this work we used a simple MLP metanetwork as a proof of concept. For editing larger model architectures a more sophisticated metanetwork architecture may be required. E.g. CNNs, Neural Functional Networks (NFNs) (Zhou et al., 2023), Transformers (Knyazev et al., 2023), or graph metanetworks (Lim et al., 2024)

Dealing with symmetries Other metanetwork architectures might also help in dealing with the many symmetries (CNN filter permutation, MLP hidden layer permutation (Zhou et al., 2023), scaling (Kalogeropoulous et al., 2024)) that exist in NN model weights. In our current approach no effort is made to either represent the models in a form that is more symmetry invariant, or to use a metanetwork architecture that is symmetry invariant. This means that to a large extent dealing with these symmetries is delegated to the metanetwork. Relieving some of this recombinatorial pressure might help the metanetwork achieve better performance with fewer parameters.

Privacy-preserving auditing. Weight Space model editing has the potential to open a new operational paradigm: *Third-party model editing*. A regulator or auditor can now receive a proprietary model, unlearn a discovered bias or vulnerability using a pretrained metanetwork, and return the sanitized model without the model owner ever disclosing their proprietary training data. This is impossible with gradient-based unlearning methods that require data access.

Legacy model retrofitting In practical deep learning life-cycles, trained models frequently outlive their underlying datasets. As Chundawat et al. (2023) observe, the availability of training data is often transient i.e. constrained by limited-duration cloud licenses, evolving access rights in data marketplaces, or strict privacy mandates like the GDPR. Thus organizations can rely on ‘orphaned’ models: valuable predictive assets whose training data is not available anymore. When regulations or users mandate concept removal, these models face obsolescence, as retraining is impossible. Our framework can resolve this by enabling compliance retrofitting directly in weight space. This allows for the sanitization of legacy assets without the need for lost data, or an attempt to reconstruct it.

9. Conclusion

We introduced data-free model editing through the lens of WSL. By training a metanetwork to accurately predict the per-class recall of a model directly from its parameters, we established a differentiable proxy for model performance that requires no access to the original training or forget data. Our empirical results on the Small CNN Zoo demonstrate that backpropagating a task-specific loss through this metanetwork allows for targeted interventions, effectively unlearning specific classes or improving performance while largely preserving functionality on control tasks.

This methodology offers a potential solution for managing models where training data is restricted, lost, or legally protected. By enabling third-party editing and privacy-preserving auditing, our approach circumvents the limitations of traditional gradient-based methods that strictly rely on data access.

Further research is required to develop this approach into a mature framework. Current limitations regarding inter-model variance and inter-class bias highlight the need for more robust experimentation. Future work should investigate larger model zoos and alternative input representations to improve the generalization and stability of weight-space model editing.

References

- Ainsworth, S. K., Hayase, J., and Srinivasa, S. Git re-basin: Merging models modulo permutation symmetries. *arXiv preprint arXiv:2209.04836*, 2022.
- Braun, D., Bushnaq, L., Heimersheim, S., Mendel, J., and Sharkey, L. Interpretability in Parameter Space: Minimizing Mechanistic Description Length with Attribution-based Parameter Decomposition. *arXiv preprint arXiv:2501.14926*, February 2025. doi: 10.48550/arXiv.2501.14926.
- Bushnaq, L., Braun, D., and Sharkey, L. Stochastic Parameter Decomposition. *arXiv preprint arXiv:2506.20790*, 2025. doi: 10.48550/arXiv.2506.20790.
- Chundawat, V. S., Tarun, A. K., Mandal, M., and Kankanhalli, M. Zero-shot machine unlearning. *IEEE Transactions on Information Forensics and Security*, 18:2345–2354, 2023.
- Eilertsen, G., Jönsson, D., Ropinski, T., Unger, J., and Ynnerman, A. Classifying the classifier: dissecting the weight space of neural networks. In *European Conference on Artificial Intelligence (ECAI 2020)*, volume 325, pp. 1119–1126. IOS PRESS, 2020.
- Falk, D., Meynert, L., Pfammatter, F., Schürholt, K., and Borth, D. A model zoo of vision transformers, 2025. URL <https://arxiv.org/abs/2504.10231>.
- Foster, J., Fogarty, K., Schoepf, S., Dugue, Z., Öztireli, C., and Brintrup, A. An information theoretic approach to machine unlearning. *arXiv preprint arXiv:2402.01401*, 2024a.
- Foster, J., Schoepf, S., and Brintrup, A. Fast machine unlearning without retraining through selective synaptic dampening. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 12043–12051, 2024b. doi: 10.1609/aaai.v38i11.29092.
- Golatkar, A., Achille, A., and Soatto, S. Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9304–9312, 2020. doi: 10.1109/CVPR42600.2020.00932.
- Ilharco, G., Ribeiro, M. T., Wortsman, M., Gururangan, S., Schmidt, L., Hajishirzi, H., and Farhadi, A. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022a.
- Ilharco, G., Wortsman, M., Gadre, S. Y., Song, S., Hajishirzi, H., Kornblith, S., Farhadi, A., and Schmidt, L. Patching open-vocabulary models by interpolating weights. volume 35, pp. 29262–29277, 2022b.
- Kalogeropoulos, I., Bouritsas, G., and Panagakis, Y. Scale Equivariant Graph Metanetworks. In *Advances in Neural Information Processing Systems*, volume 37, pp. 106800–106840. Curran Associates, Inc., 2024. doi: 10.52202/079017-3391.
- Knyazev, B., Hwang, D., and Lacoste-Julien, S. Can We Scale Transformers to Predict Parameters of Diverse ImageNet Models? In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 17243–17259. PMLR, 23–29 Jul 2023.
- Krizhevsky, A., Hinton, G., et al. Learning multiple layers of features from tiny images. 2009. URL <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- Lecun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Liang, Z., Tang, D., Zhou, Y., Zhao, X., Shi, M., Zhao, W., Li, Z., Wang, P., Schürholt, K., Borth, D., Bronstein, M. M., You, Y., Wang, Z., and Wang, K. Drag-and-Drop LLMs: Zero-Shot Prompt-to-Weights. *arXiv preprint arXiv:2506.16406*, June 2025. doi: 10.48550/arXiv.2506.16406.
- Lim, D., Maron, H., and Law, M. T. Graph Metanetworks for Processing Diverse Neural Architectures. *ICLR*, 2024.
- Mishra, A., Kumar, T., Nayak, G., Shah, A., Bhattacharya, S., and Foltin, M. Erasing clip memories: Non-destructive, data-free zero-shot class unlearning in clip models, 2025. URL <https://arxiv.org/abs/2512.14137>.
- Moulin, O., Francois-lavet, V., Elbers, P., and Hoogendoorn, M. Leveraging weights signals – predicting and improving generalizability in reinforcement learning, 2025. URL <https://arxiv.org/abs/2511.20234>.
- Navon, A., Shamsian, A., Achituve, I., Fetaya, E., Chechik, G., and Maron, H. Equivariant architectures for learning in deep weight spaces. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 25790–25816. PMLR, 23–29 Jul 2023.
- Netzer, Y., Wang, T., Coates, A., Bissacco, A., Wu, B., Ng, A. Y., et al. Reading digits in natural images with unsupervised feature learning. In *NIPS workshop on deep learning and unsupervised feature learning*, volume 2011, pp. 7, 2011. URL https://www-cs.stanford.edu/~twangcat/papers/nips2011_housenumbers.pdf.

- Nguyen, T. T., Huynh, T. T., Ren, Z., Nguyen, P. L., Liew, A. W.-C., Yin, H., and Nguyen, Q. V. H. A Survey of Machine Unlearning. *ACM Transactions on Intelligent Systems and Technology*, 16(5), 2025. ISSN 2157-6904. doi: 10.1145/3749987.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. Learning Transferable Visual Models from Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pp. 8748–8763. PMLR, 18–24 Jul 2021.
- Rangel, J. M. L., Schoepf, S., Foster, J., Krueger, D., and Anwar, U. Learning to forget using hypernetworks. *arXiv preprint arXiv:2412.00761*, 2024. doi: 10.48550/arXiv.2412.00761.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. doi: 10.48550/arXiv.1707.06347.
- Schürholt, K., Mahoney, M. W., and Borth, D. Towards scalable and versatile weight space learning. In *Proceedings of the 41st International Conference on Machine Learning*. JMLR, 2024. URL <https://proceedings.mlr.press/v235/schurholt24a.html>.
- Schürholt, K., Kostadinov, D., and Borth, D. Self-Supervised Representation Learning on Neural Network Weights for Model Characteristic Prediction. In *Advances in Neural Information Processing Systems*, volume 34, pp. 16481–16493. Curran Associates, Inc., 2021.
- Schürholt, K., Knyazev, B., Giró-i Nieto, X., and Borth, D. Hyper-Representations as Generative Models: Sampling Unseen Neural Network Weights. *Advances in Neural Information Processing Systems*, 35:27906–27920, December 2022a.
- Schürholt, K., Taskiran, D., Knyazev, B., Giró-i Nieto, X., and Borth, D. Model Zoos: A Dataset of Diverse Populations of Neural Network Models. *Advances in Neural Information Processing Systems*, 35:38134–38148, December 2022b.
- Tarun, A. K., Chundawat, V. S., Mandal, M., and Kankanhalli, M. Fast yet effective machine unlearning. *IEEE Transactions on Neural Networks and Learning Systems*, 35(9):13046–13055, 2023.
- Unterthiner, T., Keyser, D., Gelly, S., Bousquet, O., and Tolstikhin, I. Predicting Neural Network Accuracy from Weights. *arXiv preprint arXiv:2002.11448*, April 2021. doi: 10.48550/arXiv.2002.11448.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. u., and Polosukhin, I. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- Wang, K., Tang, D., Zhao, W., Schürholt, K., Wang, Z., and You, Y. Recurrent Diffusion for Large-Scale Parameter Generation. *Advances in Neural Information Processing Systems*, December 2025. doi: 10.48550/arXiv.2501.11587. URL <http://arxiv.org/abs/2501.11587>.
- Xiao, H., Rasul, K., and Vollgraf, R. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017. doi: 10.48550/arXiv.1708.07747.
- Zhou, A., Yang, K., Burns, K., Cardace, A., Jiang, Y., Sokota, S., Kolter, J. Z., and Finn, C. Permutation Equivariant Neural Functionals. *Advances in Neural Information Processing Systems*, September 2023. doi: 10.48550/arXiv.2302.14040. URL <http://arxiv.org/abs/2302.14040>.

A. More Related work

A.1. Parameter decomposition

In their work on *Attribution-based Parameter Decomposition* (Braun et al., 2025) posit that when training a neural network, the optimization algorithm is compressing the information that is relevant to the task into a networks weights. For example, a well-trained LLM will probably contain the knowledge that "the sky is blue". This should be stored somewhere in its weights. The natural central question is then: Can this information be located and separated from all other knowledge? Is there a subnetwork, a component of the parameters, which activates whenever this knowledge must be used in a forward pass, but never otherwise? The authors introduce a novel technique that shows promising signs of doing exactly this.

The **core hypothesis** of Braun et al. (2025) is that if the weight space contains atomized mechanisms (APD/SPD), then evidence of knowledge should be retrievable from the models' weights. There is an information-theoretical question that underlies this hypothesis: can the information that the model has learned be explained in full by its weights? Or does additional critical information only exist in combination with the data or architecture?

Our work intersects with this paradigm. Rather than decomposing single networks, we ask whether common mechanisms or weight-pattern "signatures" emerge across a population of networks. For instance, if many CNNs learn the same task, do they converge to similar weight components or circuits? We were inspired by the hypothesis that if weight-space mechanisms are real, evidence of them should be detectable by a metanetwork scanning across many networks

A.2. Recent developments of WSL

A.2.1. SCALING AND VERSATILITY

In recent years impressive steps have been made to address symmetries, scaling, and generalization to diverse feedforward architectures in WSL.

Zhou et al. (2023) introduce a novel architecture: Neural Function Networks (NFNs) to address permutation symmetries. Lim et al. (2024) presented a method of representing an NN as a *parameter graph*, which has the benefit of being parameter permutation symmetry-equivariant, but unlike established computation graph representations allow for efficient handling of parameter-sharing layers. These parameter graphs are modular and flexible, and Graph Neural Networks can then be trained for a range of downstream tasks on the parameter graphs, such as prediction and parameter generation. Kalogeropoulos et al. (2024) extend this methodology by also addressing scale symmetries that result from using ReLU activation functions. Schürholt et al. (2024) utilize sequential transformers to learn task-agnostic representations of models. Due to the sequential processing of neural network weights this method is very scalable. These developments suggest that, if an effective model-steering method can be established, there is reason to expect it will be scalable.

A.2.2. SELF-SUPERVISED WSL

Another approach is to use self-supervised learning (SSL) where metanetworks create *hyper-representations* of model weights. These learned representations can be used to predict hyperparameters, test accuracy, and generalization (Schürholt et al., 2021), but also as a basis for the generation of unseen NNs (Schürholt et al., 2022a).

In a similar vein, multiple other methods are proposed which train a neural network to generate new instances of NN parameters (Knyazev et al., 2023; Wang et al., 2025; Liang et al., 2025), in this context referred to as a *hypernetworks*. Knyazev et al. (2023) have utilized Transformer models (Vaswani et al., 2017). For the generation of favorable initializations of ImageNet Models. Wang et al. (2025) and Liang et al. (2025) have scaled the use of hypernetworks (metanetworks) to generate full NN parameters for sizes of up to 100 million, from LLMs to ResNets and vision transformers (ViTs).

B. Unlearning results per class

B.1. MNIST

Table 4. Unlearning: MNIST per-class results (classes 0–4). Sample sizes: 0 (814), 1 (70), 2 (4711), 3 (2922), 4 (4160).

Method	0		1		2		3		4	
	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$
Before	0.86	0.77	0.97	0.42	0.85	0.90	0.85	0.85	0.91	0.90
Retain finetune	0.00	0.78	0.82	0.42	0.06	0.90	0.00	0.86	0.00	0.91
Gradient ascent	0.00	0.21	0.00	0.17	0.00	0.13	0.00	0.11	0.00	0.11
Random vector	0.77	0.64	0.95	0.36	0.77	0.81	0.83	0.77	0.88	0.86
Our method	0.69	0.54	0.63	0.36	0.41	0.69	0.85	0.66	0.74	0.77

Table 5. Unlearning: MNIST per-class results (classes 5–9). Sample sizes: 5 (4350), 6 (715), 7 (1269), 8 (3484), 9 (4526).

Method	5		6		7		8		9	
	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$
Before	0.86	0.92	0.78	0.63	0.87	0.82	0.87	0.88	0.88	0.90
Retain finetune	0.01	0.93	0.05	0.65	0.00	0.83	0.01	0.89	0.00	0.92
Gradient ascent	0.00	0.17	0.00	0.31	0.00	0.11	0.00	0.23	0.00	0.14
Random vector	0.78	0.88	0.73	0.59	0.84	0.71	0.84	0.82	0.82	0.84
Our method	0.54	0.81	0.72	0.50	0.73	0.63	0.75	0.77	0.38	0.76

B.2. Fashion-MNIST

Table 6. Unlearning: Fashion-MNIST per-class results (classes 0–4). Sample sizes: 0 (3409), 1 (1244), 2 (4539), 3 (2436), 4 (4009).

Method	0		1		2		3		4	
	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$
Before	0.75	0.75	0.89	0.65	0.73	0.78	0.79	0.73	0.71	0.79
Retain finetune	0.00	0.77	0.08	0.66	0.00	0.81	0.01	0.75	0.00	0.83
Gradient ascent	0.00	0.21	0.00	0.13	0.00	0.25	0.00	0.11	0.00	0.12
Random vector	0.72	0.69	0.89	0.61	0.68	0.75	0.76	0.70	0.69	0.77
Our method	0.49	0.64	0.85	0.54	0.43	0.71	0.63	0.62	0.08	0.64

Table 7. Fashion-MNIST per-class results (classes 5–9). Sample sizes: 5 (5225), 6 (4118), 7 (2911), 8 (1774), 9 (2421).

Method	5		6		7		8		9	
	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$
Before	0.83	0.74	0.26	0.84	0.86	0.73	0.87	0.68	0.94	0.70
Retain finetune	0.02	0.74	0.00	0.85	0.00	0.75	0.07	0.69	0.02	0.71
Gradient ascent	0.00	0.12	0.00	0.30	0.00	0.13	0.00	0.19	0.00	0.11
Random vector	0.79	0.69	0.30	0.81	0.85	0.71	0.85	0.64	0.92	0.65
Our method	0.13	0.62	0.12	0.80	0.56	0.68	0.85	0.58	0.68	0.54

B.3. CIFAR-10

Table 8. CIFAR-10 per-class results (classes 0–4). Sample sizes: 0 (3193), 1 (2394), 2 (3346), 3 (1284), 4 (2644).

Method	0		1		2		3		4	
	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$
Before	0.39	0.37	0.50	0.37	0.22	0.39	0.16	0.44	0.35	0.39
Retain finetune	0.00	0.39	0.00	0.40	0.00	0.40	0.00	0.45	0.00	0.41
Gradient ascent	0.00	0.12	0.00	0.12	0.00	0.11	0.00	0.12	0.00	0.13
Random vector	0.33	0.32	0.47	0.33	0.15	0.33	0.14	0.38	0.28	0.34
Our method	0.03	0.34	0.15	0.35	0.05	0.34	0.02	0.38	0.13	0.35

Table 9. CIFAR-10 per-class results (classes 5–9). Sample sizes: 5 (2905), 6 (2662), 7 (2112), 8 (3374), 9 (3038).

Method	5		6		7		8		9	
	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$
Before	0.47	0.37	0.54	0.37	0.38	0.40	0.55	0.35	0.45	0.36
Retain finetune	0.00	0.40	0.00	0.40	0.00	0.41	0.00	0.39	0.00	0.39
Gradient ascent	0.00	0.11	0.00	0.13	0.00	0.11	0.00	0.11	0.00	0.12
Random vector	0.39	0.33	0.49	0.33	0.32	0.36	0.47	0.31	0.41	0.32
Our method	0.22	0.34	0.32	0.33	0.18	0.36	0.15	0.33	0.23	0.33

B.4. SVHN cropped

Table 10. Unlearning: SVHN per-class results (classes 0–4). Sample sizes: 0 (2166), 1 (709), 2 (1911), 3 (1870), 4 (1283).

Method	0		1		2		3		4	
	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$
Before	0.37	0.42	0.79	0.28	0.79	0.33	0.42	0.43	0.51	0.44
Retain finetune	0.00	0.42	0.00	0.32	0.00	0.40	0.00	0.45	0.00	0.46
Gradient ascent	0.00	0.12	0.00	0.11	0.00	0.11	0.00	0.11	0.00	0.11
Random vector	0.26	0.34	0.58	0.21	0.60	0.23	0.32	0.35	0.42	0.37
Our method	0.27	0.33	0.37	0.21	0.43	0.24	0.22	0.35	0.28	0.38

Table 11. Unlearning: SVHN per-class results (classes 5–9). Sample sizes: 5 (1793), 6 (823), 7 (1517), 8 (1289), 9 (477).

Method	5		6		7		8		9	
	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$	$\mathcal{D}_t\downarrow$	$\mathcal{D}_r\uparrow$
Before	0.56	0.41	0.28	0.52	0.55	0.42	0.27	0.49	0.31	0.61
Retain finetune	0.00	0.45	0.00	0.54	0.00	0.42	0.00	0.49	0.00	0.62
Gradient ascent	0.00	0.11	0.00	0.11	0.00	0.12	0.00	0.12	0.00	0.12
Random vector	0.48	0.36	0.24	0.46	0.42	0.37	0.19	0.40	0.24	0.51
Our method	0.51	0.35	0.19	0.46	0.35	0.37	0.17	0.40	0.16	0.53

Table 16. Uplearning: CIFAR-10 per-class results (classes 0–4). Sample sizes: 0 (722), 1 (1056), 2 (983), 3 (904), 4 (1059).

Method	0		1		2		3		4	
	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$
Before	0.38	0.36	0.42	0.33	0.18	0.36	0.06	0.36	0.33	0.34
Random vector	0.23	0.27	0.29	0.24	0.01	0.20	0.02	0.23	0.09	0.24
Our method	0.59	0.22	0.38	0.22	0.36	0.15	0.53	0.16	0.35	0.20

C. Performance improvement results per class

C.1. MNIST

Table 12. Uplearning: MNIST per-class results (classes 0–4). Sample sizes: 0 (137), 1 (4), 2 (383), 3 (315), 4 (262).

Method	0		1		2		3		4	
	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$
Before	0.50	0.51	0.77	0.17	0.61	0.76	0.78	0.71	0.51	0.69
Random vector	0.42	0.48	0.00	0.11	0.56	0.69	0.76	0.63	0.45	0.63
Our method	0.41	0.52	1.00	0.00	0.69	0.66	0.73	0.62	0.75	0.57

Table 13. Uplearning: MNIST per-class results (classes 5–9). Sample sizes: 5 (430), 6 (119), 7 (292), 8 (334), 9 (356).

Method	5		6		7		8		9	
	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$
Before	0.46	0.81	0.59	0.48	0.85	0.72	0.71	0.72	0.74	0.74
Random vector	0.38	0.74	0.61	0.45	0.84	0.65	0.68	0.66	0.71	0.67
Our method	0.63	0.71	0.53	0.48	0.86	0.62	0.78	0.64	0.73	0.62

C.2. Fashion-MNIST

Table 14. Uplearning: Fashion-MNIST per-class results (classes 0–4). Sample sizes: 0 (169), 1 (2), 2 (162), 3 (216), 4 (226).

Method	0		1		2		3		4	
	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$
Before	0.73	0.73	0.85	0.63	0.72	0.78	0.79	0.74	0.66	0.76
Random vector	0.69	0.63	0.82	0.62	0.66	0.73	0.75	0.68	0.60	0.71
Our method	0.80	0.54	0.71	0.62	0.64	0.63	0.64	0.62	0.84	0.61

Table 15. Uplearning: Fashion-MNIST per-class results (classes 5–9). Sample sizes: 5 (76), 6 (216), 7 (115), 8 (55), 9 (2).

Method	5		6		7		8		9	
	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$
Before	0.76	0.72	0.18	0.80	0.82	0.67	0.74	0.54	0.34	0.25
Random vector	0.76	0.70	0.22	0.61	0.81	0.64	0.68	0.50	0.50	0.07
Our method	0.87	0.65	0.91	0.31	0.84	0.57	0.71	0.54	0.34	0.16

Table 17. Uplearning: CIFAR-10 performance improvement per-class results (classes 5–9). Sample sizes: 5 (1069), 6 (1064), 7 (152), 8 (1064), 9 (768).

Method	5		6		7		8		9	
	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$	$\mathcal{D}_t \uparrow$	$\mathcal{D}_r$
Before	0.43	0.33	0.45	0.34	0.11	0.28	0.51	0.32	0.44	0.36
Random vector	0.16	0.23	0.27	0.24	0.00	0.17	0.36	0.26	0.37	0.28
Our method	0.84	0.15	0.59	0.19	0.65	0.10	0.78	0.19	0.65	0.24

C.3. CIFAR-10

D. Unlearning result examples

MNIST

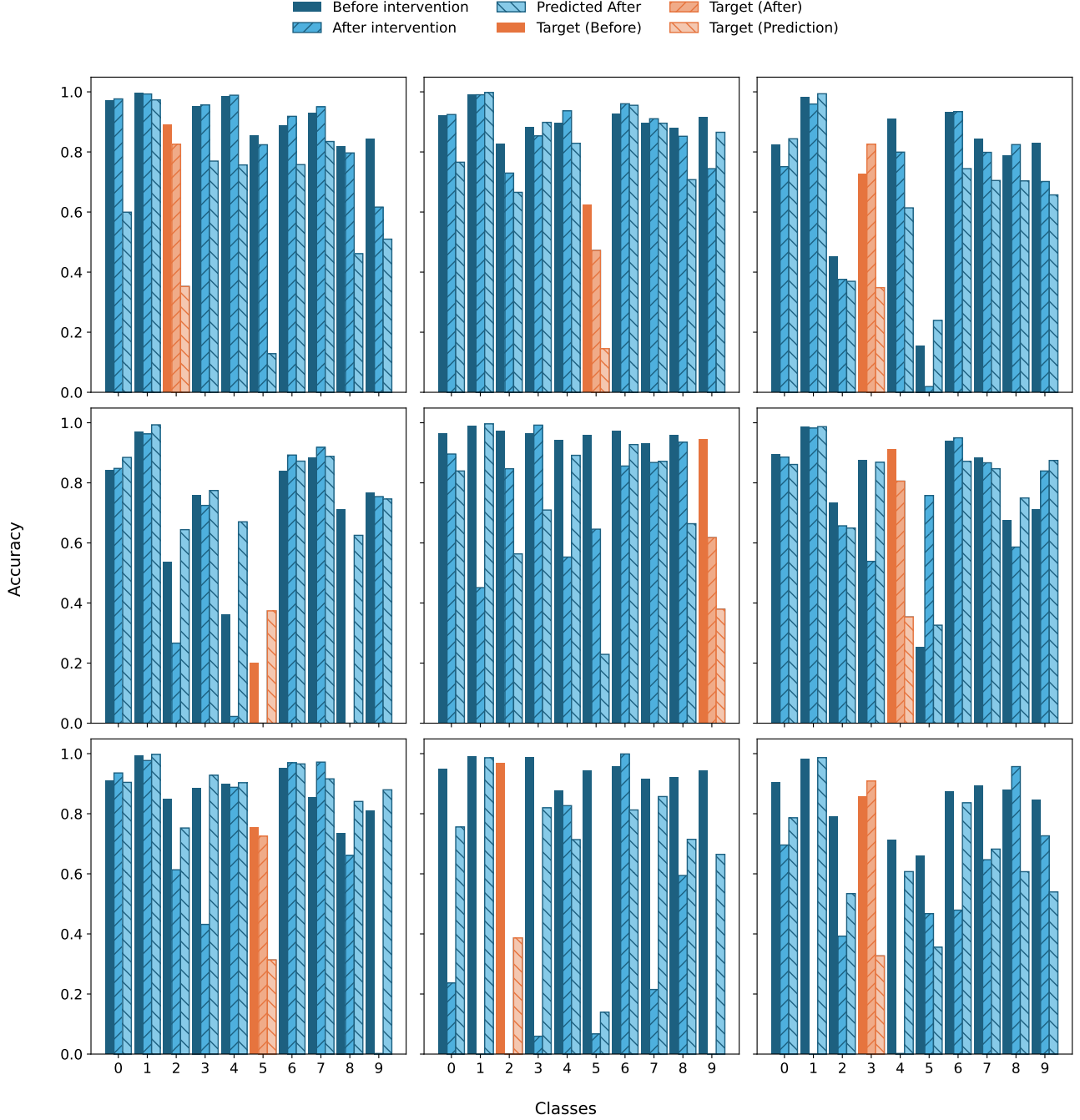


Figure 6. An unbiased sample of 9 MNIST models unlearned using our method. Barplot shows accuracies per class on the test set before intervention, after intervention and the predicted accuracies by the metanetwork after intervention. Target class in orange.

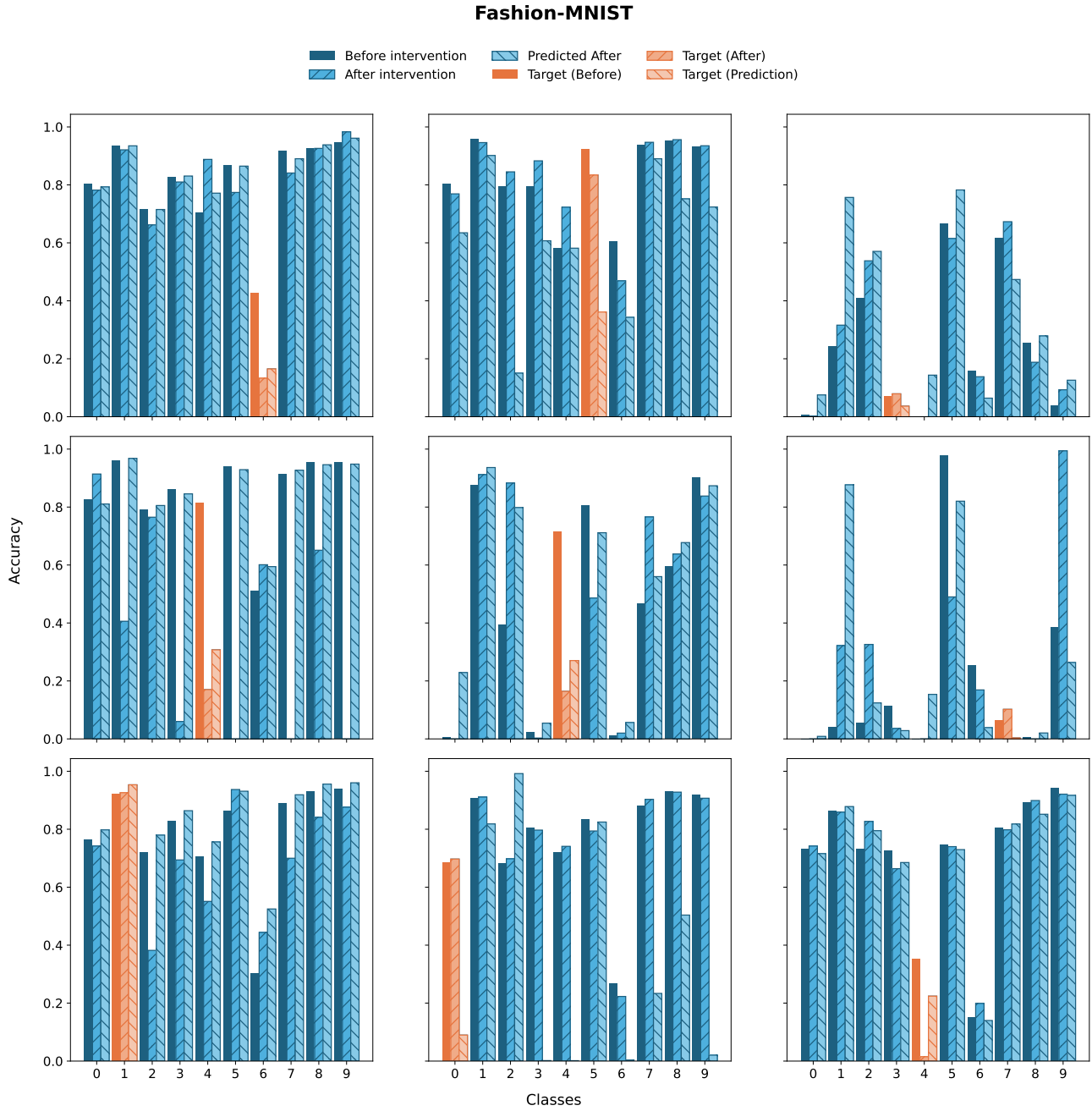


Figure 7. An unbiased sample of 9 Fashion MNIST models unlearned using our method. Barplot shows accuracies per class on the test set before intervention, after intervention and the predicted accuracies by the metanetwork after intervention. Target class in orange.

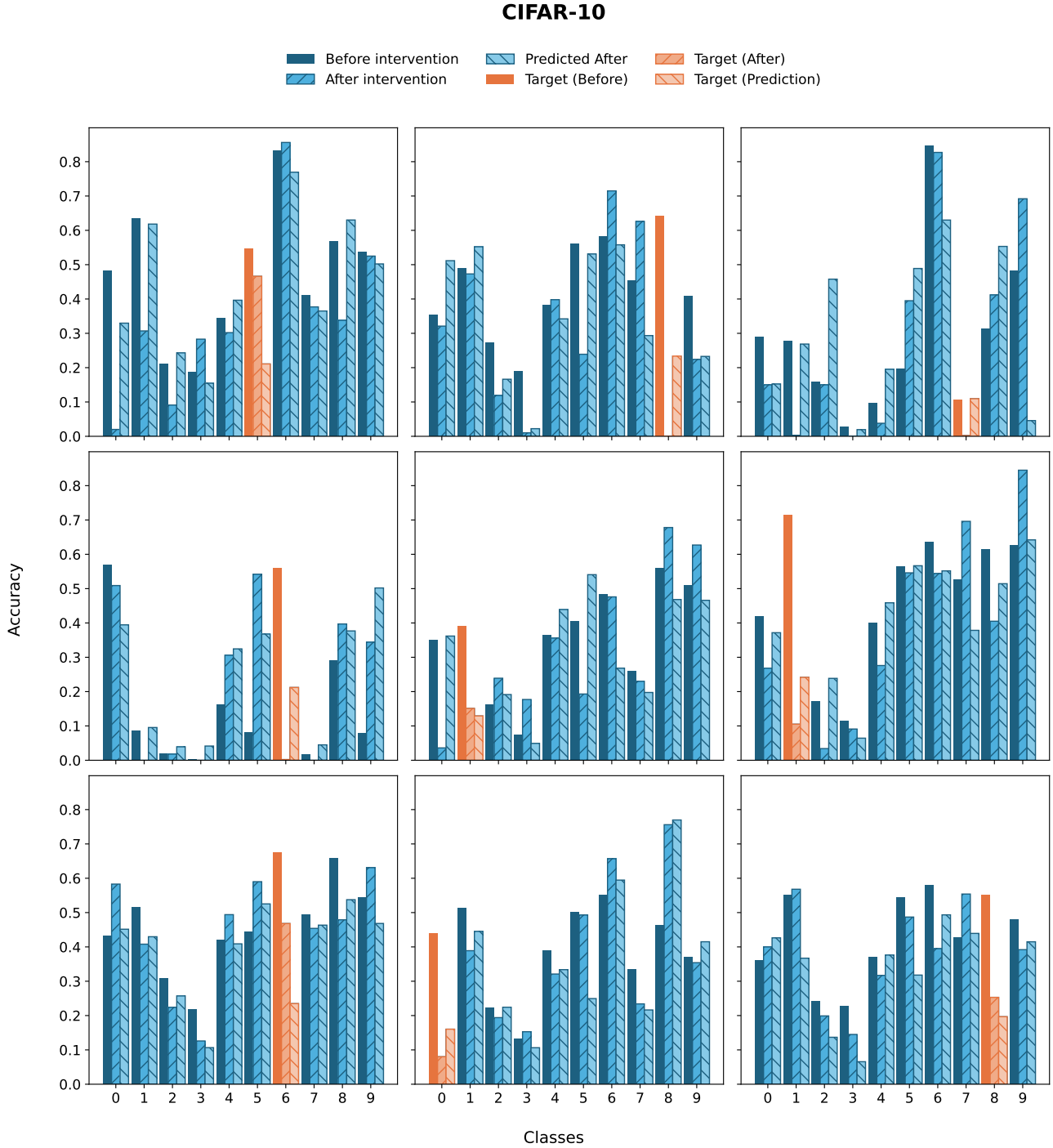


Figure 8. An unbiased sample of 9 CIFAR-10 models unlearned using our method. Barplot shows accuracies per class on the test set before intervention, after intervention and the predicted accuracies by the metanetwork after intervention. Target class in orange.

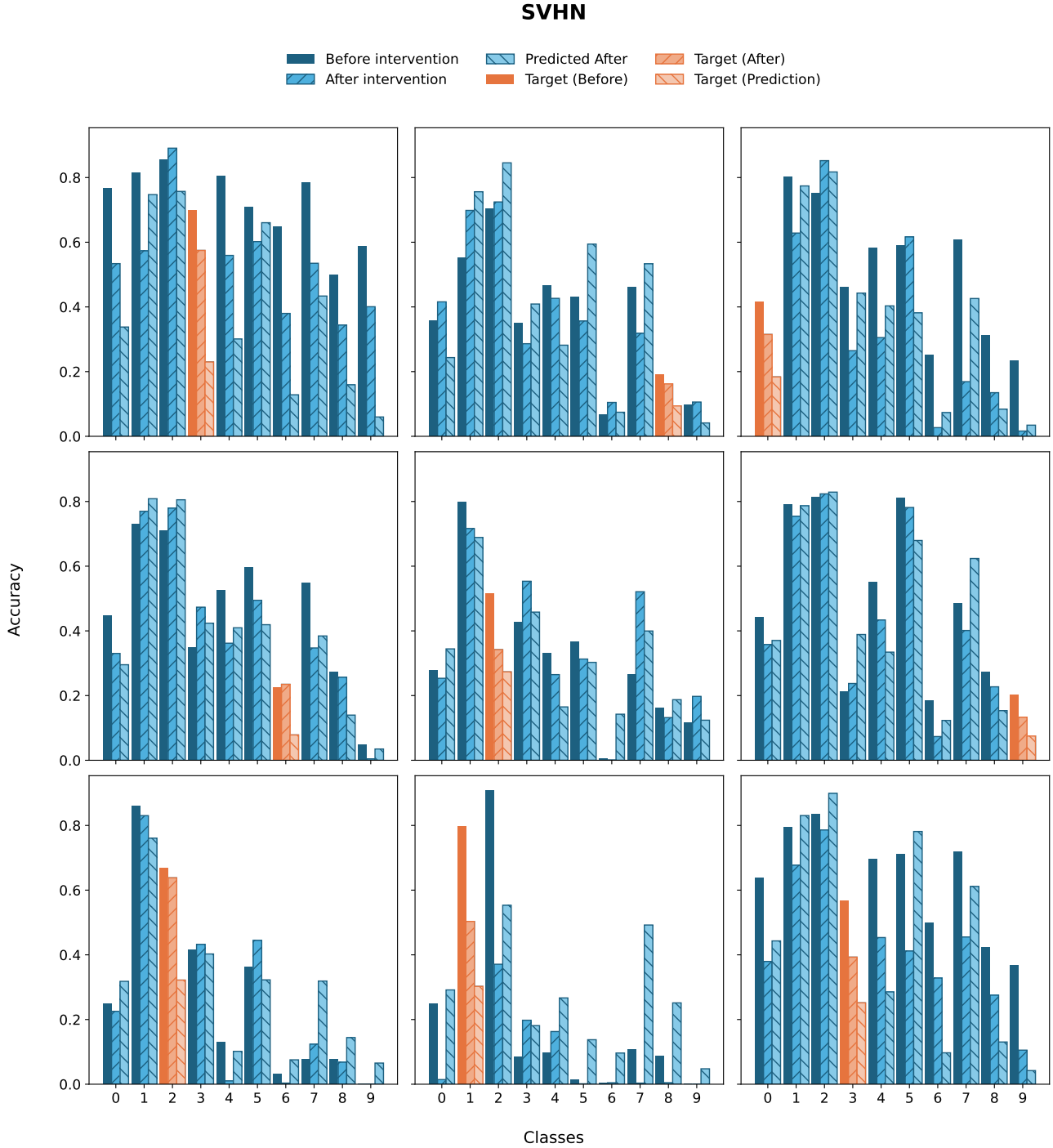


Figure 9. An unbiased sample of 9 SVHN models unlearned using our method. Barplot shows accuracies per class on the test set before intervention, after intervention and the predicted accuracies by the metanetwork after intervention. Target class in orange.

E. Unfiltered Data

1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099

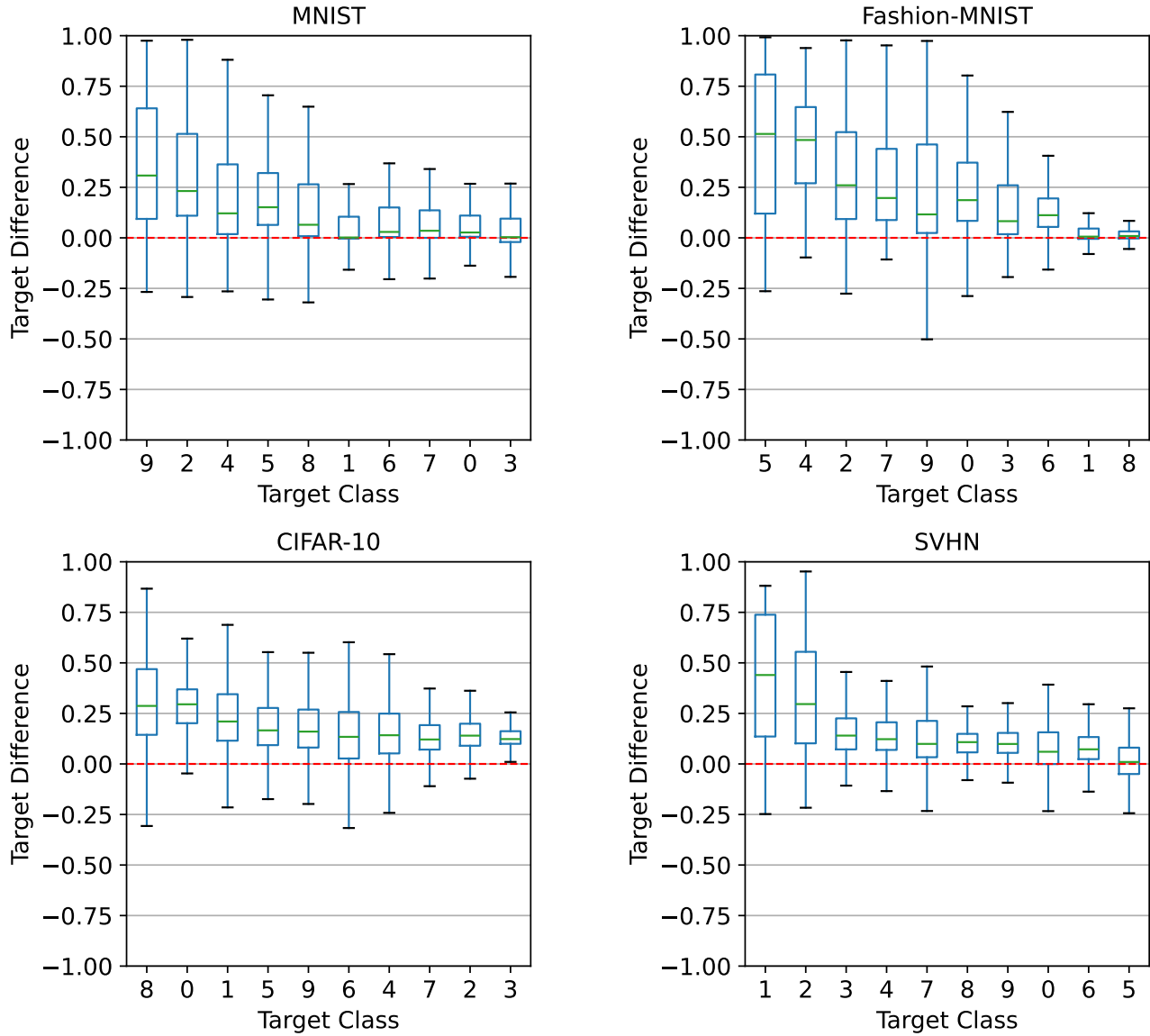


Figure 10. Boxplots of target difference by target class after intervention for **unfiltered** CNNs MNIST, Fashion-MNIST, CIFAR-10, and SVHN on the test set. Ordered by mean target difference; outliers omitted. Upwards of the red dotted line marks successful intervention.

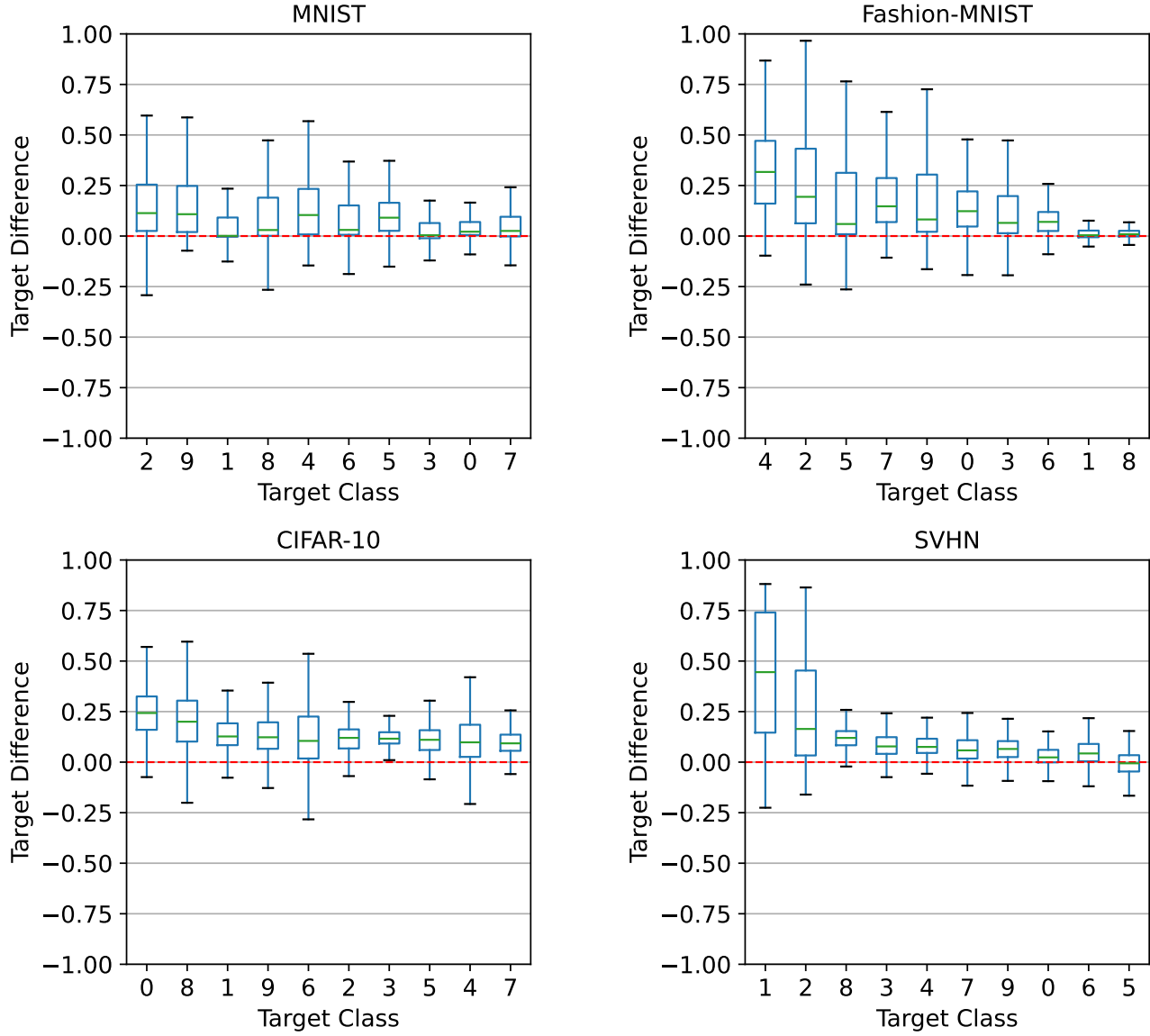


Figure 11. Boxplots of target difference by target class after intervention for **only filtered** CNNs MNIST, Fashion-MNIST, CIFAR-10, and SVHN on the test set. Ordered by mean target difference; outliers omitted. Upwards of the red dotted line marks successful intervention.

F. Extending the experimentation

The most common criticism voiced by reviewers was about the extent of the experimentation. They believed:

1. Our metanetwork was trained on a zoo of very small 'toy' CNNs trained on simple classification datasets. The field of weight space learning has matured in the past few years to the extent that many downstream tasks have been performed proficiently on larger computer vision models (Larger CNNs and ResNets)².
2. Comparison with benchmarks was not extensive enough. We should preferably choose unlearning methodologies which would be a fair comparison to our method by working with the same assumptions and constraints.

F.1. Larger base models

A good candidate metanetwork for testing our methodology on larger base models could be SANE (Schürholt et al., 2024). In order to achieve success the following steps must be completed:

1. Reason whether unlearning via gradients w.r.t. the input can work on SANE (specifically given the tokenization of the weights, and that not all weights are incorporated in a forward pass, but rather a smaller 'window').
2. Evaluate the models the models in the CIFAR-10 CNN model zoo and save the per-class recall scores. These models have 3 convolution and 2 dense layers and 12,000 parameters per model, already 2.4x increase in CNN size compared to our previous experiments.
3. Pretrain a SANE autoencoder on this dataset.
4. Train a prediction head on this SANE model instance to predict per-class recall
5. Perform unlearning via Algorithm 1.

There are many steps at which this plan can fail. I've got some alternatives in my head but let's get to those when it turns out to be needed.

²And recently a ViT Model Zoo with the purpose of weight space learning has been released (Falk et al., 2025). However, research on weight space learning on Transformer architectures is still very early stage.